Missing data



Social and Cultural Analytics (7AAVBCS5)
Dr Mark J. Hill · mark.j.hill@kcl.ac.uk
2026-10-19

Rescehduled lecture

We’ve rescheduled the missed lecture for **November 11, 9:00-11:00 in STR 242.**

There will be additional workshop hours over revision week, and I will extend my office hours for those who cannot make those.

Lecture Plan

- **Part I: Missing data**
  - What counts as missing
  - `NaN`, `None`, and `.isna()`
  - Why pandas can be quiet about gaps
- **Part II: Missingness in London cultural data**
  - Counts and percentages
  - Website, postcode, and address fields
  - Missingness as evidence
- **Part III: Strategies**
  - Delete, fill, or keep
  - Missingness indicators
  - Preserving the original data
- **Part IV: Joining context**
  - How borough context gets attached
  - Checking whether missingness is patterned
- **Part V: Live coding**

Key Learning Outcomes

By the end of this week you will be able to:

- identify missing values in pandas
- quantify missingness with counts and percentages
- explain why missingness may be meaningful
- choose a defensible missing-data strategy
- create missingness indicators
- join venue records to borough context
- interpret missingness with denominators and caution

Review

What we've learned so far

- Python as a calculator
- Data types (`int`, `float`, `str`, `bool`)
- How to assign variables
- Manipulating/operating on variables
- Notebook basics
- Lists, indexing and slicing
- Arrays (numpy) and array-wise operations
- Basic descriptive stats
  - min, max, sum, mean, median, standard deviation
- Dictionaries
- f-strings
- Comparisons and `if`/`elif`/`else`

What we learned last week

- Data frames store rows and columns
- One row should describe one thing
- `pd.read_csv()` loads CSV files
- `.info()`, `.head()`, `.shape`, `.columns`, and `.describe()` inspect structure
- Square brackets select columns
- `.loc[]` selects by label
- `.iloc[]` selects by position
- Masks filter rows
- `to_csv(index=False)` saves data without writing row labels

This week: what happens when a table has gaps

Part I: Missing Data

Missing data

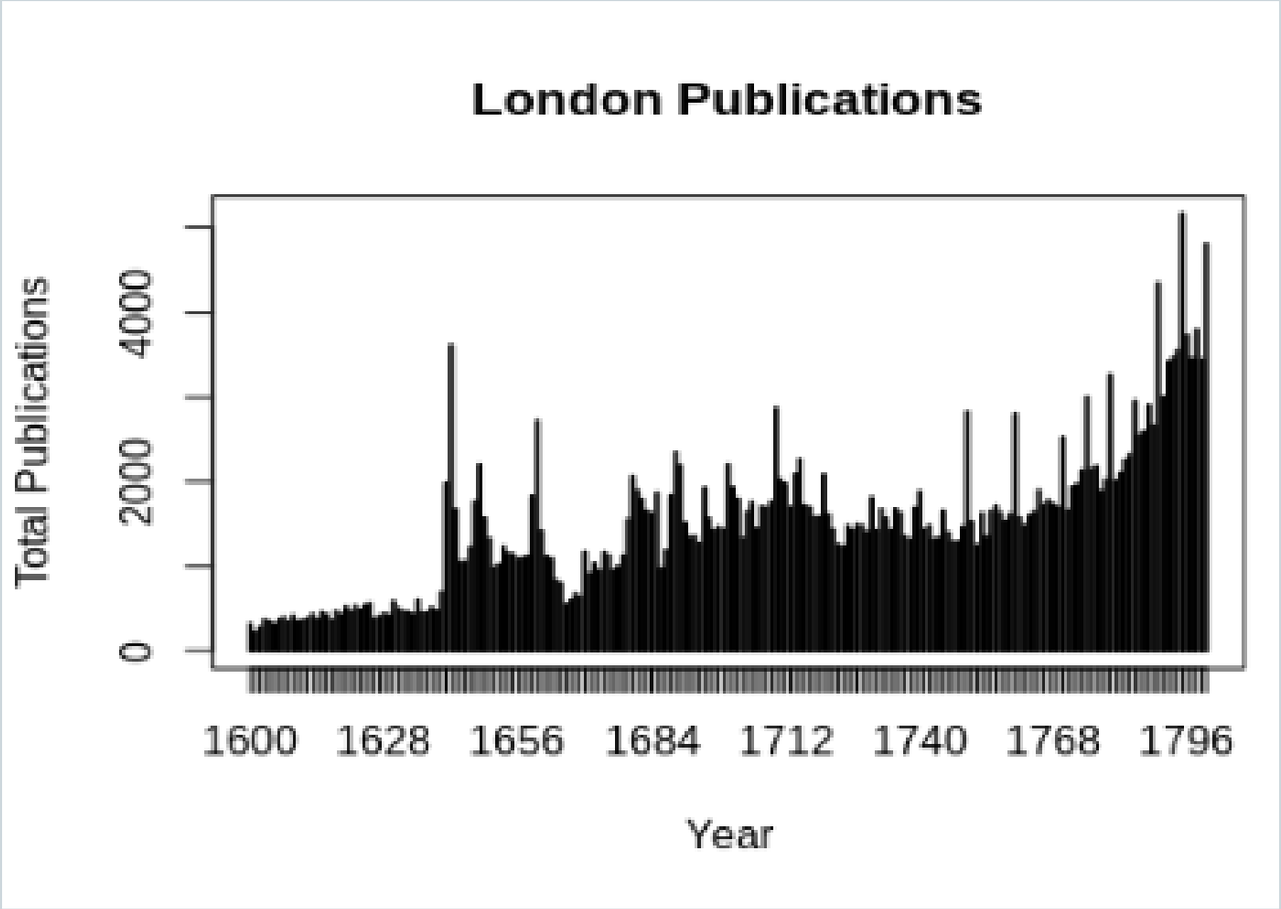
What is missing data?

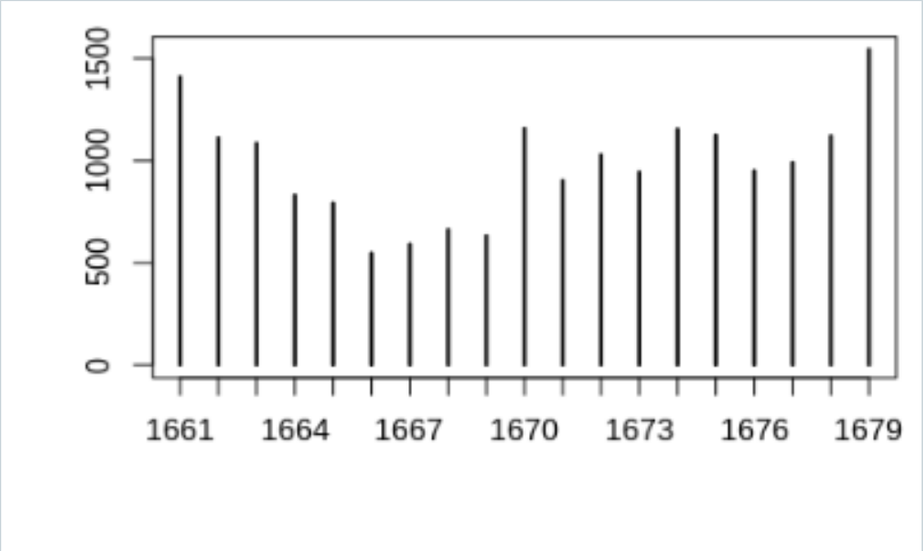
- a value that should logically exist, but does not
- a blank in a public record
- a field skipped by a survey respondent
- a place that cannot be joined to another geography because a postcode is absent
- an unknown value created by earlier data collection choices

Why it matters

- summaries may use fewer rows than you think
- comparisons may exclude particular places or groups
- missingness can reveal how the dataset was made

A historical gap: London, 1666





ESTC publication records show a marked fall around 1666.

The Great Fire disrupted London's publishing infrastructure. The gap is evidence of a historical event, not a value to fill in.

Source: English Short Title Catalogue (ESTC)

Why values go missing

Cause	Example	Question to ask
Recording error	a field was left blank during data entry	Is the error scattered or concentrated?
Non-response	someone does not answer an income question	Who is more likely not to answer?
Impossible to record	a historical source no longer exists	What historical process produced the gap?
By design	a field does not apply to some rows	Is this really missing, or not applicable?

When a gap is about provenance

Object type	Region	Acquisition method	Original owner	Cultural significance
Pottery	Pacific Islands	Purchase	not recorded	Ceremonial
Jewellery	Southeast Asia	Colonial collection	not recorded	not recorded
Sculpture	West Africa	Colonial collection	not recorded	not recorded
Pottery	South America	Donation	not recorded	Domestic
Mask	Pacific Islands	Purchase	not recorded	Sacred

- `original_owner` is not recorded for **134 of 150** objects
- `cultural_significance` is not recorded for **75 of 150** objects
- `acquisition_method` is not recorded for **43 of 150** objects

Illustrative teaching dataset; not evidence about a named museum.

Investigating historical missingness: Lubartów

Investigating missingness: Lubartów, Poland (1932–1943) (Lemercier and Zalc, 2026)

A population register recorded residents' religion, but 911 of 11,950 entries had no religion recorded.

- **Explanation 1: Clerical error**. Missing religion was associated with other missing fields and entries written in pencil.
- **Explanation 2: Deliberate concealment**. Religion was more likely to be missing for subtenants (people renting rooms from another tenant) and less likely for property owners. People with few formal ties to a property may have found it easier to conceal Jewish identity from authorities during Nazi persecution.

Key point: By analysing **whose** information was missing, researchers could investigate whether gaps reflected administrative errors or attempts to escape persecution.

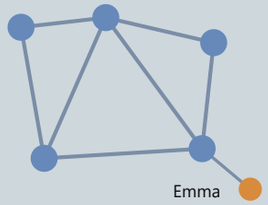
Quantification helped evaluate competing historical explanations for missing data. It did not establish with certainty why each individual entry was blank.

When sources disagree: Queen Emma’s social networks

A study reconstructed the social networks of **Emma, a tenth-century queen of France**, using different historical sources.

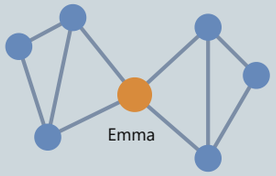
- **Narrative accounts:** Emma appeared relatively peripheral to the networks of political elites.
- **Legal deeds:** Emma appeared as an important bridge between her father’s family and her husband’s family.

Narrative sources



Emma appears peripheral to the recorded elite relationships.

Legal deeds



Emma appears as a bridge between her father’s and husband’s families.

Conceptual illustrations, not reproductions of the study’s actual networks.

Neither source necessarily provides the “correct” network. Each records relationships that mattered to its authors and purposes. **Key point:** Contradictory representations can reveal how different sources construct our understanding of historical relationships. Rather than merging the networks into one supposedly complete dataset, researchers compared them to investigate these differences.

Missingness may be a research question

Instead of asking only “how do we handle or remove blanks?”, ask:

- Which fields are absent, and how does that impact (especially limit) our question?
- Could the gaps reflect the source or collection process?
- Which records would disappear if we insisted on complete data?

Missing values in pandas

```
import numpy as np
import pandas as pd

venues = pd.read_csv("../../data/london_cultural_venues_2023.csv")

# Five real venue records with missing website and/or postcode values.
example_venues = venues.loc[
    [917, 2094, 1482, 1620, 1032, 1065],
    ["name", "website", "postcode"],
]

example_venues
```

	name	website	postcode
917	Jubilee Gardens Library	NaN	NaN
2094	Lexington (The)	https://www.thelexington.co.uk/	NaN
1482	Shacklewell Arms	www.shacklewellarms.com	71 Shacklewell Lan
1620	Sainsbury Theatre	https://www.lamda.ac.uk/hire-our-spaces	NaN
1032	Commonwealth Studios	NaN	NaN
1065	Rose and Crown	NaN	SE10 8ER

```
example_venues.isna()
```

	name	website	postcode
917	False	True	True
2094	False	False	True
1482	False	False	False
1620	False	False	True
1032	False	True	True
1065	False	True	False

In pandas, missing values often appear as `NaN` or `None`. Use `.isna()` to detect both.

`NaN`, `None`, and empty strings

Value	Plain meaning	How to detect it
`NaN`	missing or undefined numeric value	`.isna()`
`None`	no Python value supplied	`.isna()`
`""`	an empty string, not automatically missing	compare to `""` or clean first
`0`	a real value, unless the codebook says otherwise	do not treat as missing by default

Missing values are not the same as zeros or empty text. Check the codebook and the data source.

Quick note: Do not compare to missing values

```
1  print(np.nan == np.nan)
2  print(example_venues["website"] == np.nan)
```

```
False
917     False
2094    False
1482    False
1620    False
1032    False
1065    False
Name: website, dtype: bool
```

```
1  example_venues["website"].isna()
```

```
917      True
2094    False
1482    False
1620    False
1032     True
1065     True
Name: website, dtype: bool
```

Never test for missing values with `== np.nan`. Use `.isna()`.

pandas can be quiet about gaps

```
example_locations = venues.loc[
    [0, 1, 2, 1645],
    ["name", "easting"],
]

example_locations
```

	name	easting
0	DAGENHAM RM10	549626.1016
1	Ice House Court	544093.0124
2	River Mills Studios (Bow Arts)	544071.1313
1645	Omega Works	NaN

```
print("mean easting:", round(example_locations["easting"].mean(), 1))

print("based on:", example_locations["easting"].notna().sum(), "of", len(example_locations), "venues")
```

```
mean easting: 545930.1
based on: 3 of 4 venues
```

pandas summary methods usually skip missing values by default.

That can be useful, but it also means nothing may warn you that a result uses fewer rows than the full dataset.

The practical (for Python) issue

Missing data affects every later step:

- a summary can be based on fewer records than we realise
- joining records may exclude those with missing fields
- ignoring them (or deleting) can remove the cases we most need to understand

The venue records

```
venues = pd.read_csv("../../data/london_cultural_venues_2023.csv")

venues[[
    "name", "venue_type", "borough_name", "postcode", "website",
    "latitude", "longitude",
]].head()
```

	name	venue_type	borough_name	postcode	website	latitude	longitude
0	DAGENHAM RM10	Artists workspaces	Barking and Dagenham	NaN	https://www.arbeit.org.uk/dagenham	51.549708	0.156633
1	Ice House Court	Artists workspaces	Barking and Dagenham	NaN	www.bowarts.org www.artiststudiofinder.org ...	51.531488	0.075994
2	River Mills Studios (Bow Arts)	Artists workspaces	Barking and Dagenham	Barking	https://bowarts.org/studios/riverside-mills-st...	51.532525	0.075726
3	Vue Dagenham	Cinemas	Barking and Dagenham	RM9 6UQ	NaN	51.532095	0.135759
4	Bow Arts	Creative workspaces	Barking and Dagenham	Abbey Road	www.bowarts.org www.artiststudiofinder.org ...	51.531410	0.077946

One row represents one cultural venue.

First check: where are the gaps?

```
missing_summary = pd.DataFrame({
    "missing": venues.isna().sum(),
    "missing_%": (venues.isna().mean() * 100).round(1),
})

missing_summary[missing_summary["missing"] > 0].sort_values(
    "missing", ascending=False
)
```

	missing	missing_%
postcode	3042	68.6
address_2	2217	50.0
website	760	17.1
address_1	440	9.9
address	355	8.0
uprn	331	7.5
easting	8	0.2
northing	8	0.2
ward_code	6	0.1
ward_name	6	0.1

`data.isna()` marks missing values as `True`. `.sum()` counts those `True` values; `.mean()` gives their proportion. `pd.DataFrame({...})` builds a table from named columns, and `.sort_values(...)` orders its rows by one column.

Part II: Decisions About Missing Data

What to do when you find missing data

Ask three questions:

1. **How much is missing?**
    - Count it and calculate a percentage. Quantify! Is it a little or big problem?
2. **Why might it be missing?**
    - Random error, a pattern, a lost source, or not applicable? If systematic you need to investigate the cause.
3. **What is the task?**
    - Descriptive: Document and report
    - Predictive: May need sophisticated imputation

There is no universal “clean” version of a dataset. What follows are some strategies that depend on your answer to the above questions.

Types of missingness (MCAR, MAR, MNAR)

Term	Plain-English meaning	Example	Impact
Missing Completely At Random (**MCAR**)	a technical accident unrelated to the data	Random survey results lost in post	Missingness does not systematically bias which cases are observed, but deleting records reduces the available information
Missing At Random (**MAR**)	a *pattern* related to something we can observe	Younger respondents skip income questions	Need to account for patterns when handling
Missing Not At Random (**MNAR**)	a *pattern* related to the unobserved value itself	High earners decline to report income	Most problematic - simple methods introduce bias

The type of missingness affects our interpretation:

- Historical gaps often MNAR (power dynamics, deliberate omissions)
- Survey non-response often MAR (demographic patterns)
- Technical failures usually MCAR (truly random)

MCAR can be tested with Little's MCAR test.

Strategy 1: targeted selection

```
1  has_website = venues["website"].notna()
2  directory_venues = venues[has_website]
3
4  print("all venue records:", len(venues))
5  print("records for a web directory:", len(directory_venues))
```

```
all venue records: 4434
records for a web directory: 3674
```

`data["column"].notna()` creates a Boolean mask: `True` where the value is present and `False` where it is missing. `data[mask]` keeps only the rows where the mask is `True`. `len(data)` counts rows.

Later, you may meet `.dropna()` as a pandas shortcut for removing missing values. We will use masks here because they make the selection rule visible.

- **Pros**: easy!
- **Cons**: lose data, potentially introduce biases
- Good when the field is essential for this specific output
- Risky when it removes records for unrelated reasons
- Always state what was excluded

Strategy 2: imputation is an estimate

Imputation replaces a missing value with a **plausible estimate**, not a recovered fact.

Kind of field	Possible estimate	Main risk
Numeric	mean or median	changes the distribution
Categorical	most common category (mode)	makes the majority look larger
Within a group	a group-specific estimate	still reduces real variation

- **Pros:** keeps rows; simple to apply
- **Cons:** adds estimated data; can hide patterns of absence

Strategy 3: keep and document the gap

```
venues["has_website"] = venues["website"].notna()
venues["has_postcode"] = venues["postcode"].notna()
venues["website_status"] = "Website recorded"
venues.loc[
    venues["website"].isna(), "website_status"
] = "No website recorded"
```

```
venues.loc[
    [0, 917, 1339, 2395],
    ["name", "website", "website_status", "has_website", "postcode", "has_postcode"],
]
```

	name	website	website_status	has_website	postcode	has_postcode
0	DAGENHAM RM10	https://www.arbeit.org.uk/dagenham	Website recorded	True	NaN	False
917	Jubilee Gardens Library	NaN	No website recorded	False	NaN	False
1339	Bar A Bar	www.barabar.info/	Website recorded	True	133-135 Stoke Newi	True
2395	greengrassi	https://www.greengrassi.com/	Website recorded	True	NaN	False

`.notna()` and `.isna()` return `True` or `False` for each value. `data.loc[rows, "column"] = value` changes one named column for the selected rows.

- Keep the original field and add a label for display or a Boolean indicator for analysis.
- If possible, preserve not only the original values, but also **where they came from**: different sources may record different versions of the same thing.

Cultural implications

Reminder! When data is missing, ask:

1. Whose activities, places, or histories are less likely to be recorded?
2. What collection decisions might have produced the gap?
3. Would deletion or imputation make an existing absence less visible?

Part III: A Small pandas Toolkit

Today’s workflow

0. Load data!
1. Inspect what is in the table
2. Make a purposeful working table
3. Find and describe gaps
4. Add useful derived columns
5. Create task-specific outputs
6. Add borough context and check the join

Load the data

```
venues = pd.read_csv(
    "../../data/london_cultural_venues_2023.csv"
)

venues.head()
print("shape:", venues.shape)
print("columns:", list(venues.columns))
venues.info()
```

```
shape: (4434, 20)
columns: ['venue_id', 'name', 'venue_type', 'venue_type_raw', 'borough_name', 'borough_code', 'ward_name', 'ward_code', 'address', 'address_1', 'address_2', 'postcode', 'website', 'latitude', 'longitude', 'easting', 'northing', 'uprn', 'source_year', 'data_capture_period']
<class 'pandas.DataFrame'>
RangeIndex: 4434 entries, 0 to 4433
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   venue_id             4434 non-null   str
 1   name                 4434 non-null   str
 2   venue_type           4434 non-null   str
 3   venue_type_raw       4434 non-null   str
 4   borough_name         4434 non-null   str
 5   borough_code         4434 non-null   str
 6   ward_name            4428 non-null   str
 7   ward_code            4428 non-null   str
 8   address              4079 non-null   str
 9   address_1            3994 non-null   str
 10  address_2            2217 non-null   str
 11  postcode             1392 non-null   str
 12  website              3674 non-null   str
 13  latitude             4434 non-null   float64
 14  longitude            4434 non-null   float64
 15  easting              4426 non-null   float64
 16  northing             4426 non-null   float64
 17  uprn                 4103 non-null   str
 18  source_year          4434 non-null   int64
 19  data_capture_period  4434 non-null   str
dtypes: float64(4), int64(1), str(15)
memory usage: 1.6 MB
```

`pd.read_csv("path/to/file.csv")` reads a comma-separated file and returns a DataFrame. The string is the file path; `../` means “go up one folder” in a relative path.

Create a new column

```
# 1. Put the new column name inside square brackets.
# 2. Put a starting value after the equals sign.
venues["research_note"] = pd.NA

venues[["name", "research_note"]].head()
```

	name	research_note
0	DAGENHAM RM10	<NA>
1	Ice House Court	<NA>
2	River Mills Studios (Bow Arts)	<NA>
3	Vue Dagenham	<NA>
4	Bow Arts	<NA>

`data["new_column"] = value` adds a column to a DataFrame. `pd.NA` gives every row an empty, missing value to start with.

What categories are in the venue dataset?

```
print("boroughs:", venues["borough_name"].nunique())
print("venue types:", venues["venue_type"].nunique())

print(venues["venue_type"].unique())
```

```
boroughs: 33
venue types: 26
<ArrowStringArray>
[           'Artists workspaces',                      'Cinemas',
           'Creative workspaces',      'Dance performance venues',
       'Dance rehearsal studios',            'Fashion and design',
                    'Libraries',                  'Makerspaces',
      'Making and manufacturing',  'Museums and public galleries',
       'Music rehearsal studios',              'Music venues all',
                'Textile design',                       'Theatre',
    'Theatre rehearsal studios',                      'Archives',
                 'Arts centres', 'Creative coworking desk space',
              'Jewellery design',       'Prop and costume making',
       'Music recording studios',   'Set and exhibition building',
       'Music venues grassroots',              'Music nightclubs',
          'Commercial galleries',                  'LGBTQ venues']
Length: 26, dtype: str
```

`column.nunique()` counts distinct non-missing values. `column.unique()` returns the distinct values themselves. `print()` displays a value or result in the notebook output.

Count records in one category

```
venue_type_counts = venues["venue_type"].value_counts()
print(venue_type_counts.head(10))
print()
borough_counts = venues["borough_name"].value_counts()
print(borough_counts.head(10))
```

```
venue_type
Archives                        544
Music venues all                371
Libraries                       347
Commercial galleries            300
Artists workspaces              270
Theatre                         266
Museums and public galleries    253
Dance rehearsal studios         252
Theatre rehearsal studios       217
Dance performance venues        196
Name: count, dtype: int64

borough_name
Westminster               653
Camden                    469
Hackney                   403
Southwark                 307
Tower Hamlets             295
Lambeth                   252
Islington                 228
Lewisham                  148
```

`column.value_counts()` counts how often each recorded value occurs and sorts the largest counts first. `.head(10)` shows the first ten rows of that result.

Make a purposeful working table

```
venue_work = venues[[
    "name", "venue_type", "borough_name",
    "website", "postcode", "latitude", "longitude",
]]

venue_work.head()
```

	name	venue_type	borough_name	website	postcode	latitude	longitude
0	DAGENHAM RM10	Artists workspaces	Barking and Dagenham	https://www.arbeit.org.uk/dagenham	NaN	51.549708	0.156633
1	Ice House Court	Artists workspaces	Barking and Dagenham	www.bowarts.org www.artiststudiofinder.org ...	NaN	51.531488	0.075994
2	River Mills Studios (Bow Arts)	Artists workspaces	Barking and Dagenham	https://bowarts.org/studios/riverside-mills-st...	Barking	51.532525	0.075726
3	Vue Dagenham	Cinemas	Barking and Dagenham	NaN	RM9 6UQ	51.532095	0.135759
4	Bow Arts	Creative workspaces	Barking and Dagenham	www.bowarts.org www.artiststudiofinder.org ...	Abbey Road	51.531410	0.077946

`data[["column_a", "column_b"]]` selects several named columns and returns a DataFrame containing those columns.

Find missing values

```
1  missing_values = venue_work.isna()
2  missing_values.head()
```

	name	venue_type	borough_name	website	postcode	latitude	longitude
0	False	False	False	False	True	False	False
1	False	False	False	False	True	False	False
2	False	False	False	False	False	False	False
3	False	False	False	True	False	False	False
4	False	False	False	False	False	False	False

`data.isna()` returns a True/False table with the same rows and columns as the original. `True` marks a missing value.

Count missing values

```
1  missing_counts = missing_values.sum()
2  missing_counts
```

```
name              0
venue_type        0
borough_name      0
website         760
postcode       3042
latitude          0
longitude         0
dtype: int64
```

`.sum()` treats `True` as 1 and `False` as 0, so it counts missing values in each column.

Calculate missing percentages

```
missing_proportions = missing_values.mean()
missing_percentages = missing_proportions * 100
missing_percentages = missing_percentages.round(1)
missing_percentages
```

```
name             0.0
venue_type       0.0
borough_name     0.0
website         17.1
postcode        68.6
latitude         0.0
longitude        0.0
dtype: float64
```

`.mean()` gives the proportion of `True` values. Multiply by 100 for percentages, then use `.round(1)` to show one decimal place.

Summarise gaps

```
missing_summary = pd.DataFrame({
    "missing": missing_counts,
    "missing_%": missing_percentages,
})

missing_summary = missing_summary.sort_values("missing", ascending=False)
missing_summary
```

	missing	missing_%
postcode	3042	68.6
website	760	17.1
name	0	0.0
borough_name	0	0.0
venue_type	0	0.0
latitude	0	0.0
longitude	0	0.0

`pd.DataFrame({...})` makes a table from named columns. `.sort_values(...)` orders the rows from the largest missing count to the smallest.

Add derived columns

```
venue_work["has_website"] = venue_work["website"].notna()
venue_work["has_postcode"] = venue_work["postcode"].notna()
venue_work["map_ready"] = (
    venue_work["latitude"].notna()
    & venue_work["longitude"].notna()
)

venue_work[["name", "has_website", "has_postcode", "map_ready"]].head()
```

	name	has_website	has_postcode	map_ready
0	DAGENHAM RM10	True	False	True
1	Ice House Court	True	False	True
2	River Mills Studios (Bow Arts)	True	True	True
3	Vue Dagenham	False	True	True
4	Bow Arts	True	True	True

`data["new_column"] = expression` creates a column. `.notna()` returns `True` where a value is present; `&` means “and” when combining two Boolean columns. `.head()` shows the first five rows.

Count the indicators

```
1  print(venue_work["has_website"].value_counts())
2  print()
3  print(venue_work["map_ready"].value_counts())
```

```
has_website
True     3674
False     760
Name: count, dtype: int64

map_ready
True    4434
Name: count, dtype: int64
```

`column.value_counts()` works for Boolean columns too: it counts the `True` and `False` values.

Select rows for each task

```
directory_mask = venue_work["website"].notna()
directory_venues = venue_work[directory_mask]

map_mask = venue_work["map_ready"]
map_venues = venue_work[map_mask]

print("all venues:", len(venue_work))
print("directory venues:", len(directory_venues))
print("map venues:", len(map_venues))
```

```
all venues: 4434
directory venues: 3674
map venues: 4434
```

`data[mask]` selects the rows where a Boolean mask is `True`. Here, `directory_mask` comes from `notna()` and `map_mask` uses the `map_ready` column created earlier. `len(data)` counts rows.

Make a readable display label

```
directory_display = venue_work.drop(columns=["latitude", "longitude"])
directory_display["website_status"] = directory_display["website"].fillna(
    "No website recorded"
)
directory_display.head()
```

	name	venue_type	borough_name	website	postcode	has_website	has_postcode	map_ready	
0	DAGENHAM RM10	Artists workspaces	Barking and Dagenham	https://www.arbeit.org.uk/dagenham	NaN	True	False	True	https://www.arbeit.org.u
1	Ice House Court	Artists workspaces	Barking and Dagenham	www.bowarts.org www.artiststudiofinder.org ...	NaN	True	False	True	www.bowarts.org ww ...
2	River Mills Studios (Bow Arts)	Artists workspaces	Barking and Dagenham	https://bowarts.org/studios/riverside-mills-st...	Barking	True	True	True	https://bowarts.org/stud
3	Vue Dagenham	Cinemas	Barking and Dagenham	NaN	RM9 6UQ	False	True	True	No website recorded
4	Bow Arts	Creative workspaces	Barking and Dagenham	www.bowarts.org www.artiststudiofinder.org ...	Abbey Road	True	True	True	www.bowarts.org ww ...

`data.drop(columns=["a", "b"])` returns a DataFrame without named columns. `.fillna(value)` returns a column with missing values replaced by `value`; assigning it to a new column preserves the original.

Sort to inspect borough context

```
borough_context = pd.read_csv(
    "../../data/london_cultural_infrastructure_2023_borough_context.csv"
)

borough_density = borough_context[[
    "borough_name", "venues_per_10k_residents",
]]

borough_density = borough_density.sort_values(
    "venues_per_10k_residents", ascending=False
)
borough_density.head()
```

	borough_name	venues_per_10k_residents
10	City of London	137.04
0	Westminster	28.32
1	Camden	20.36
2	Hackney	15.34
4	Tower Hamlets	10.38

`data.sort_values("column", ascending=False)` sorts rows from largest to smallest by one column. Use `ascending=True` for smallest to largest; `.head()` then shows the first rows of the sorted table.

Mark high-access boroughs

```
1  borough_context["high_ptal"] = borough_context["ptal_score_2014"] >= 5
2  borough_context["high_ptal"].value_counts()
```

```
high_ptal
False    26
True      7
Name: count, dtype: int64
```

`data["new_column"] = condition` creates a Boolean column from a comparison. `value_counts()` then counts the `True` and `False` values.

Part IV: Adding Borough Context

Two tables, two units

Table	One row represents	Example question
`venues`	one cultural venue	Which venue types are recorded?
`borough_context`	one London borough	Which boroughs have high PTAL?

The shared key is `borough_name`.

Read a left join in parts

1. Start with `venue_work` on the **left**: one row represents one venue.
2. Use `borough_context` on the **right**: one row represents one borough.
3. `on="borough_name"` tells pandas to match rows where the borough name is the same in both tables.
4. `how="left"` keeps every venue row, even if a borough has no matching context row.

Before joining	After a left join
one row per venue	still one row per venue
borough context is separate	matching borough values are attached

A left join

```
context_columns = [
    "borough_name",
    "median_household_income_2012_13",
    "ptal_score_2014",
    "venues_per_10k_residents",
    "high_ptal",
]

context_for_join = borough_context[context_columns]

venues_with_context = venue_work.merge(
    context_for_join,
    on="borough_name",
    how="left",
)
```

`left.merge(right, on="key", how="left")` matches rows using a shared key. `on=` names that key; `how="left"` keeps every row from the DataFrame on the left.

Always check a join

```
print("rows before:", len(venue_work))
print("rows after:", len(venues_with_context))

missing_ptal = venues_with_context["ptal_score_2014"].isna()
print("missing PTAL after join:", missing_ptal.sum())
```

```
rows before: 4434
rows after: 4434
missing PTAL after join: 0
```

`len(data)` counts rows. `column.isna()` makes a True/False result; `.sum()` then counts the `True` values.

> A join can multiply rows or leave unmatched records. Check it.

A bounded question with new context

```
high_ptal_venue_types = venues_with_context.loc[
    venues_with_context["high_ptal"], "venue_type"
]

high_ptal_venue_type_counts = high_ptal_venue_types.value_counts()
high_ptal_venue_type_counts.head(10)
```

```
venue_type
Archives                        401
Commercial galleries            240
Music venues all                199
Museums and public galleries    144
Theatre                         137
Theatre rehearsal studios       112
Dance rehearsal studios         107
Dance performance venues        104
Jewellery design                 79
Music nightclubs                 74
Name: count, dtype: int64
```

`data["new_column"] = condition` creates a Boolean column from a comparison. `.loc[mask, "column"]` keeps one named column for rows where the Boolean `mask` is `True`.

Part V: Live Coding

Live-coding workflow

Question: How usable is the venue dataset for an online directory and a map?

1. Inspect venue categories and counts
2. Make a focused working table
3. Audit missing values
4. Add availability and map-readiness columns
5. Create purposeful outputs
6. Add borough context and check the result

For the worksheet: select case-study boroughs

```
selected_boroughs = ["Camden", "Hackney"]

case_venues = venues[
    venues["borough_name"].isin(selected_boroughs)
]
```

`column.isin(values)` returns `True` where a column value appears in a list. Put that Boolean result inside `data[...]` to keep matching rows.